

实验二 原生应用开发

实验项目基本信息

- 实验编号：d20301035102、d20301009202
- 学时分配：4学时
- 实验类型：验证型
- 每组人数：6人
- 对应课程目标：课程目标1

核心步骤列表

实验概览

- 需求分析阶段：编写登录功能需求文档（输入验证规则、页面跳转逻辑）
- AI辅助编码阶段：使用TRAE生成登录页面骨架代码并完善业务逻辑
- 测试验证阶段：编写输入验证测试用例，验证边界条件
- 部署运维阶段：打包调试版本，记录平台差异对比报告

开发任务清单

任务阶段	主要内容	关键技术点
Android 登录	Kotlin + Activity 跳转	EditText、Intent、Bundle
主页面设计	校园卡功能实现	RecyclerView、CardView
校园卡界面	信息展示与交互	ViewModel、LiveData

✓ 成功标准

- Android 登录页面功能完整
- Activity 跳转与数据传递正常
- 输入验证逻辑正确
- 主页面与校园卡界面展示正常

实验任务与案例

核心任务

使用Kotlin实现Android登录页面（EditText输入验证、Activity跳转与数据传递），并完成主页面和校园卡界面的设计与实现。

实战案例

开发"智慧校园"登录模块，学生在登录页面输入学号和密码，系统验证成功后跳转到个人中心页面，显示个性化的欢迎信息和用户资料。

第一课时：Android登录页面设计（2学时）

2.1 需求分析阶段

步骤1：编写需求文档

1. 功能需求

用户名输入（学号）

- 密码输入
- 登录按钮
- 输入验证（不能为空、长度限制）

2. 界面需求

Material Design风格

- 输入框占位提示
- 密码隐藏显示

3. 跳转需求

登录成功 → 个人中心

- 传递用户名数据

2.2 Android项目创建

步骤1 : 创建项目

1. 打开Android Studio
2. 创建Empty Activity项目
3. 选择Kotlin语言
4. 最低SDK: API 24

2.3 AI辅助编码

步骤1 : 生成登录页面代码

1. 使用TRAE描述需求: Kotlin登录页面, EditText输入验证, Activity跳转
2. AI生成代码片段
3. 手动完善业务逻辑

步骤2 : 实现布局

```
<!-- activity_login.xml -->
<LinearLayout
    android:orientation="vertical"
    android:padding="16dp"

    <EditText
        android:id="@+id/et_username"
        android:hint="请输入学号"
        android:inputType="text"/
    >

    <EditText
        android:id="@+id/et_password"
        android:hint="请输入密码"
        android:inputType="textPassword"/
    >

    <Button
        android:id="@+id/btn_login"
        android:text="登录"/
    >
</LinearLayout>
```

步骤3 : 实现登录逻辑

```
// LoginActivity.kt
class LoginActivity : AppCompatActivity() {
```

```
private lateinit var etUsername: EditText
private lateinit var etPassword: EditText
private lateinit var btnLogin: Button

override fun onCreate(savedInstanceState: Bundle?)
{
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_login)

    etUsername = findViewById(R.id.et_username)
    etPassword = findViewById(R.id.et_password)
    btnLogin = findViewById(R.id.btn_login)

    btnLogin.setOnClickListener {
        val username = etUsername.text.toString()
        val password = etPassword.text.toString()

        if (validateInput(username, password))
        {
            login(username, password)
        }
    }
}

private fun validateInput(username: String, password: String): Boolean
{
    if (username.isEmpty()) {
        etUsername.error = "学号不能为空"
        return false
    }
    if (password.isEmpty())
    {
        etPassword.error = "密码不能为空"
        return false
    }
    return true
}

private fun login(username: String, password: String) {
    // 模拟登录验证
    if (username == "admin" && password == "123456") {
        // 跳转到个人中心
        val intent = Intent(this, HomeActivity::class.java)
        intent.putExtra("username", username)
        startActivity(intent)
    } else {
        Toast.makeText(this, "用户名或密码错误", Toast.LENGTH_SHORT).show()
    }
}
```

```
}  
}
```

2.4 创建个人中心页面

步骤1：创建HomeActivity

1. 新建Activity: HomeActivity
2. 布局显示欢迎信息

步骤2：接收传递数据

```
// HomeActivity.kt  
class HomeActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?)  
    {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_home)  
  
        val username = intent.getStringExtra("username")  
        val tvWelcome = findViewById<TextView>(R.id.tv_welcome)  
        tvWelcome.text = "欢迎 $username"  
    }  
}
```

第二课时：主页面与校园卡界面设计（2 学时）

2.5 主页面设计

步骤 1：创建主页面布局

使用 RecyclerView 展示校园服务列表，包括校园卡、成绩查询、课程表等功能入口。

```
<!-- activity_main.xml -->  
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/rv_services"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:padding="16dp"/>
```

步骤 2：实现校园卡界面

使用 CardView 展示校园卡信息，包括余额、消费记录等。

```
<!-- activity_campus_card.xml -->
<androidx.cardview.widget.CardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardCornerRadius="8dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:text="校园卡"
            android:textSize="20sp"
            android:textStyle="bold"/>

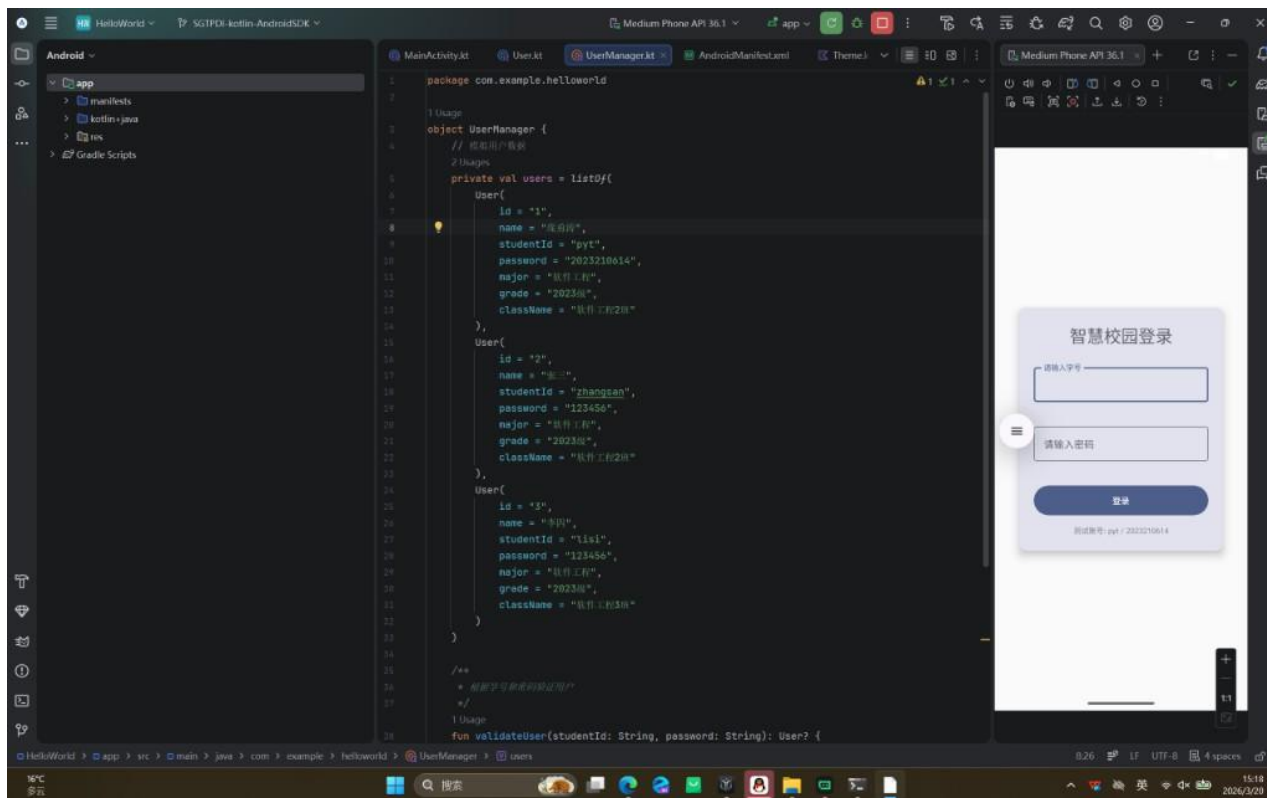
        <TextView
            android:id="@+id/tv_balance"
            android:text="余额: 100.00 元"
            android:textSize="16sp"
            android:layout_marginTop="8dp"/>

        <TextView
            android:id="@+id/tv_last_consume"
            android:text="最近消费: 早餐 8.00 元"
            android:textSize="14sp"
            android:layout_marginTop="4dp"/>
    </LinearLayout>
</androidx.cardview.widget.CardView>
```

2.6 界面截图展示

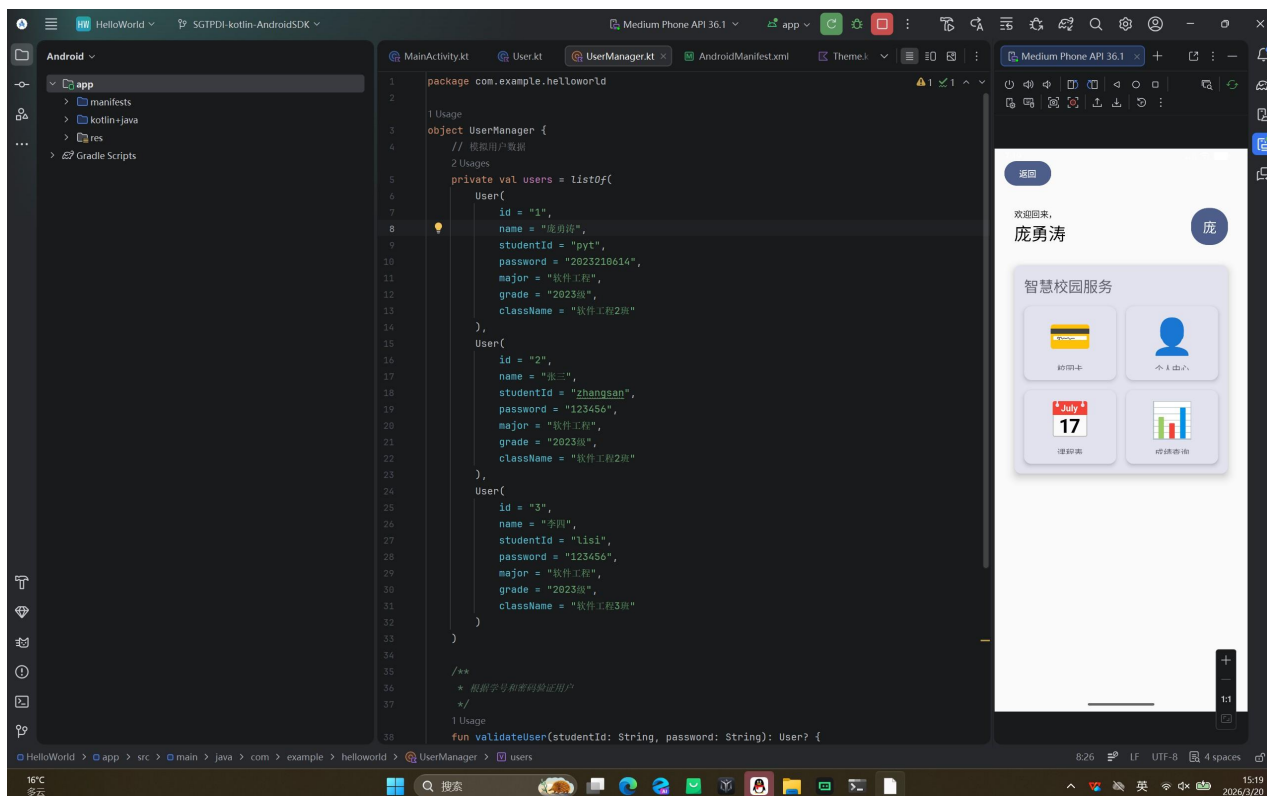
登录界面

智慧校园登录页面，包含学号输入框、密码输入框和登录按钮，采用 Material Design 风格。



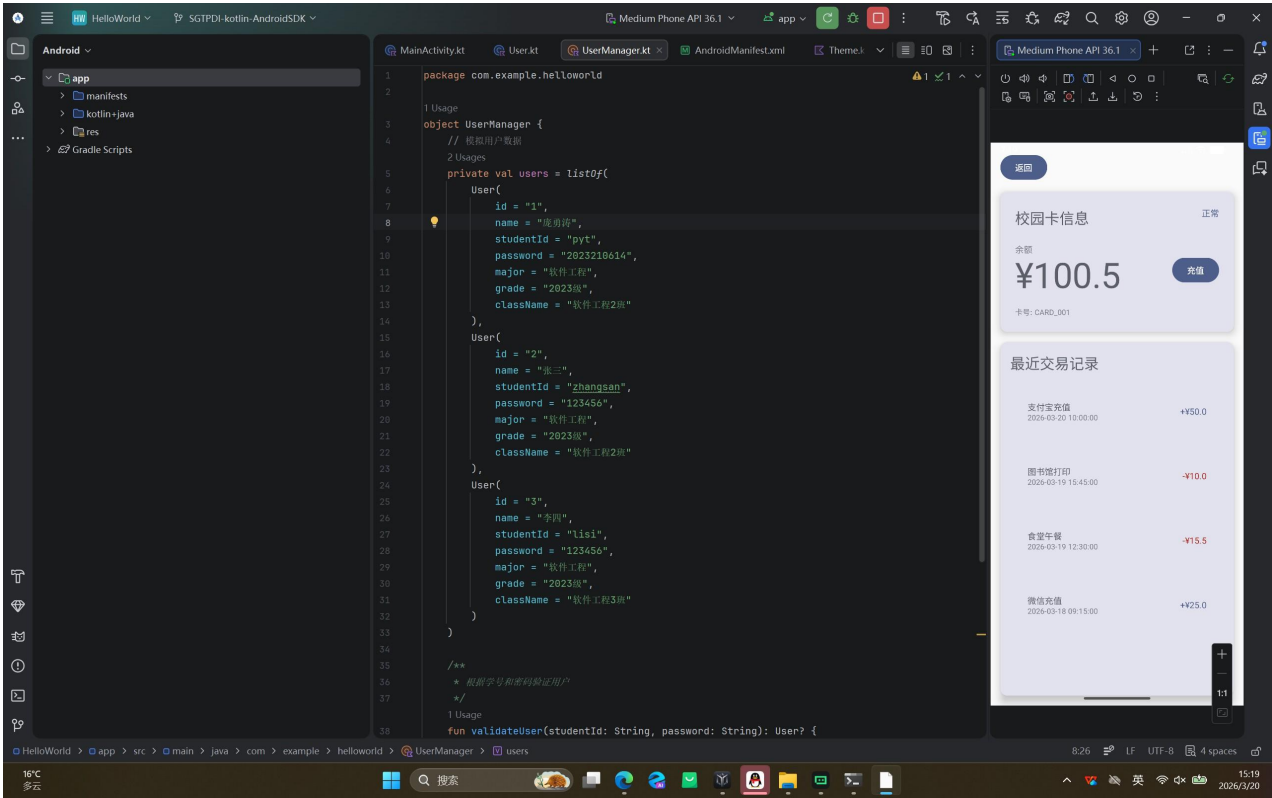
主页面

登录成功后的主界面，显示欢迎信息和四个功能入口：校园卡、个人信息、课程表和成绩查询。



校园卡界面

校园卡详情页面，显示卡余额（¥100.5）、卡号信息以及最近交易记录列表。



本次实现内容

1. 登录功能

- 学号和密码输入
- 登录验证
- 登录成功跳转
- 输入验证提示

2. 主页面

- 欢迎信息展示
- 功能卡片布局
- 校园卡入口
- 个人信息入口
- RecyclerView 列表

3. 校园卡功能

- 余额显示

- ◆ 卡号信息
- ◆ 交易记录查询
- ◆ CardView 布局

技术栈

- ◆ 开发语言: Kotlin
- ◆ UI 框架: Android View 系统
- ◆ 架构模式: MVC
- ◆ 数据传递: Intent
- ◆ UI 组件: RecyclerView、CardView
- ◆ 设计规范: Material Design

总结与思考

实验总结

掌握Android Kotlin登录页面开发

- ◆ 学会Activity间数据传递
- ◆ 完成主页面与校园卡界面设计
- ◆ 理解RecyclerView和CardView的使用
- ◆ 实现了智慧校园应用的核心功能
- ◆

课后思考

1. 如何优化RecyclerView的性能
2. 校园卡界面的数据更新机制
3. AI辅助代码生成对开发效率的影响
4. 如何进一步优化应用的用户体验
5. 如何实现更复杂的功能模块